

7. Инструменти за разработка, дизайн и стил на Flutter.

Графичен потребителски интерфейс. Управление на състоянието (State) на Flutter приложение. Разработка на мултиплатформени мобилни приложения.

1. Увод

Flutter е open-source UI framework, разработен от Google, за изграждане на native компилирани, мултиплатформени приложения от една база код (single codebase) - за Android, iOS, Web, Windows, macOS и Linux. Написан е на езика Dart - обектно-ориентиран, компилиран, с поддръжка на JIT (Just-In-Time) компилация по време на разработка и AOT (Ahead-Of-Time) компилация за production.

Основната философия на Flutter е **"everything is a widget"** - целият потребителски интерфейс, включително оформлението (layout), стиловете и дори анимациите, се изгражда чрез композиция на widget-и.

2. Инструменти за разработка

2.1 Flutter SDK и Dart SDK

Flutter SDK включва Dart SDK, rendering engine, framework библиотеки (Material, Cupertino), и инструменти за компилация. Инсталацията и

състоянието на средата се проверяват с командата:

```
flutter doctor
```

Тя проверява наличието на Android toolchain, Xcode (за iOS/macOS), свързани устройства и IDE плъгини.

2.2 IDE и редактори

- **Android Studio / IntelliJ IDEA** - пълна поддръжка чрез Flutter и Dart плъгини, вграден емулятор manager.
- **Visual Studio Code** - лек, с разширения Flutter и Dart, вградена поддръжка за дебъгване и hot reload.

2.3 Команден ред (Flutter CLI)

```
flutter create my_app      # създава нов проект
flutter run                # стартира приложението
flutter build apk          # компилира release APK за Android
flutter build ios          # компилира release build за iOS
flutter pub get            # изтегля зависимости от pubspec.yaml
flutter test               # изпълнява тестове
```

2.4 Flutter DevTools

Уеб-базиран инструмент за дебъгване и профилиране, включващ:

- **Widget Inspector** - визуализация и инспекция на widget дървото;
- **Performance view** - анализ на честотата на кадрите (frame rate);
- **Memory view** - следене на паметта и обектите (garbage collection);
- **Network view** - мониторинг на HTTP заявки.

2.5 Hot Reload и Hot Restart

Hot Reload инжектира променения код директно в работещата Dart VM, без да губи текущото състояние на приложението - ускорява цикъла на разработка. **Hot Restart** пресъздава приложението от нулата, но по-бързо от пълно пресъздаване, като запазва компилираните артефакти.

3. Дизайн и стил

3.1 Material Design и Cupertino

Flutter предоставя два готови комплекта widget-и, следващи различни дизайн модели:

- Material widgets (`MaterialApp` , `Scaffold` , `AppBar`) - следват Material Design на Google, типичен за Android.
- Cupertino widgets (`CupertinoApp` , `CupertinoNavigationBar`) - имитират iOS-стила на Apple.

3.2 Теми (Theming)

Стилът на приложението се конфигурира централно чрез `ThemeData` , което позволява консистентен визуален език без дублиране на код:

```
MaterialApp(  
  theme: ThemeData(  
    colorScheme: ColorScheme.fromSeed(seedColor: Colors.teal),  
  ),  
  textTheme: const TextTheme(  
    titleLarge: TextStyle(fontSize: 22, fontWeight: FontWeight.bold),  
  ),  
  useMaterial3: true,  
),  
home: const HomePage(),  
);
```

3.3 Адаптивен и responsive дизайн

За различни размери екрани се използват `MediaQuery` (достъп до размерите на екрана) и `LayoutBuilder` (адаптиране спрямо наличното пространство на родителския widget), както и гъвкави widget-и като `Expanded`, `Flexible` и `Wrap`.

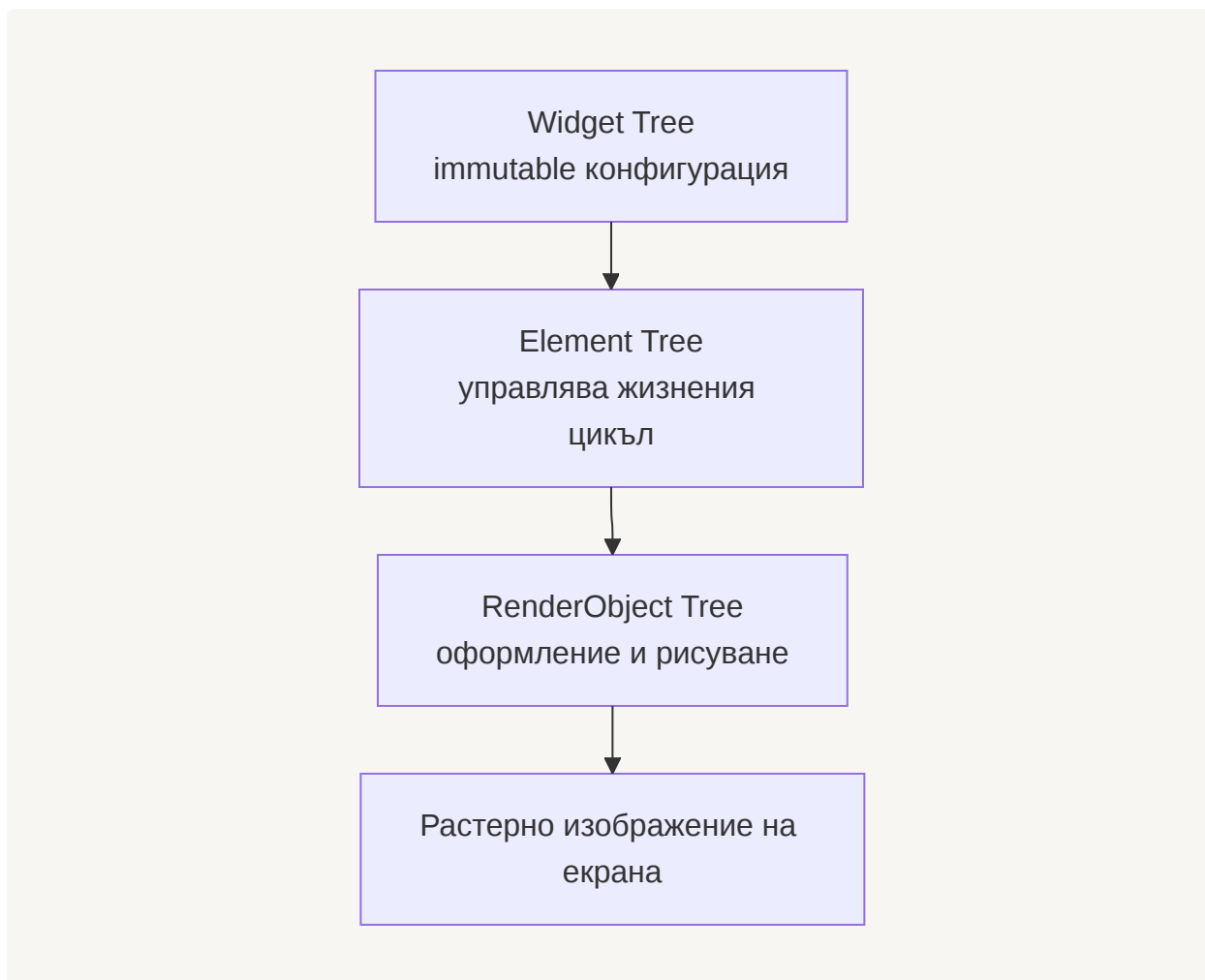
4. Графичен потребителски интерфейс (GUI)

4.1 "Everything is a widget"

Всеки визуален елемент - текст, бутон, отстъп, дори самото оформление - е widget. Widget-ите се композират в дърво (widget tree), което Flutter превежда във вътрешни структури за реално рендиране.

4.2 Трите дървета

Flutter поддържа три успоредни дървовидни структури:



Widget е неизменяемо (immutable) описание на конфигурацията на UI.

Element е "живият" екземпляр, който свързва widget-а с неговото място в дървото и управлява презаписването при промяна. **RenderObject** извършва реалните изчисления за оразмеряване, позициониране и рисуване (layout & paint).

4.3 Stateless vs Stateful Widgets

- **StatelessWidget** - Използва се за статичен интерфейс, който зависи единствено от конфигурационните си параметри, предадени при неговото конструиране. Няма вътрешно състояние, което да се променя във времето.
- **StatefulWidget** - Притежава съпътстващ обект от клас State. Използва се, когато интерфейсът трябва да реагира на потребителски събития,

мрежови заявки или вътрешни таймери, променяйки визуализацията си динамично.

```
// Widget без вътрешно състояние
class Greeting extends StatelessWidget {
  final String name;
  const Greeting({super.key, required this.name});

  @override
  Widget build(BuildContext context) {
    return Text('Здравей, $name!');
  }
}

// Widget с вътрешно, променящо се състояние
class Counter extends StatefulWidget {
  const Counter({super.key});
  @override
  State<Counter> createState() => _CounterState();
}

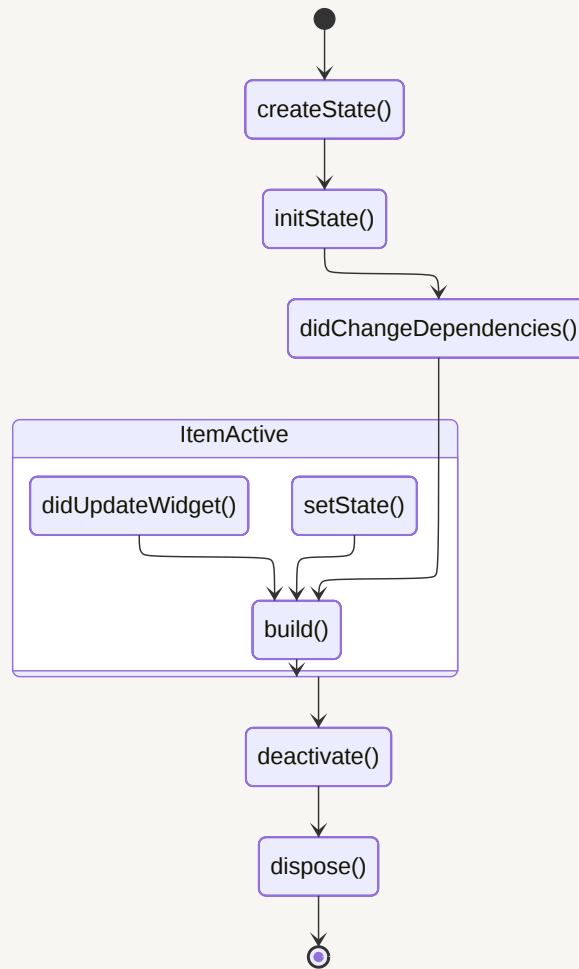
class _CounterState extends State<Counter> {
  int _count = 0;

  void _increment() => setState(() => _count++);

  @override
  Widget build(BuildContext context) {
    return Column(
      children: [
        Text('Брой: $_count'),
        ElevatedButton(onPressed: _increment, child: const Text('+')),
      ],
    );
  }
}
```

```
}  
}
```

Жизнен цикъл на `StatefulWidget` :



- `createState()` : Извиква се веднага при вмъкване на виджета в дървото. Създава асоциирания `State` обект.
- `initState()` : Извиква се точно веднъж след създаването на състоянието. Подходящо място за инициализация на променливи, анимационни контролери или абонаменти.
- `didChangeDependencies()` : Извиква се веднага след `initState()` и винаги, когато обект, от който виджета зависи (напр. `InheritedWidget` или `Theme`), се промени.

- `build()` : Основен метод. Извиква се многократно. Връща конфигурацията на виджетите, които трябва да се рендерират.
- `didUpdateWidget()` : Извиква се, ако родителският виджет промени конфигурацията си и изисква преназначаване със същия `runtimeType` и `key` .
- `deactivate()` : Извиква се, когато елементът се премахва временно от дървото.
- `dispose()` : Извиква се при перманентно премахване на обекта. Използва се за освобождаване на ресурси (затваряне на стриймове, спиране на контролери), за да се предотвратят течове на памет.

4.4 Основни layout widgets

- `Container` - обединява padding, margin, декорация и ограничения на размера;
- `Row` / `Column` - линейно оформление по хоризонтал / вертикал
- `Stack` - позициониране едно върху друго
- `ListView` / `GridView` - лист или решетъчно представяне на елементи

5. Управление на състоянието (State Management)

Едно от най-важните архитектурни решения при Flutter е как се управлява и разпространява състоянието между widget-ите.

5.1 Локално състояние - `setState()`

Най-простият механизъм (демонстриран по-горе) - извикването на `setState()` уведомява framework-a, че state-ът се е променил, и предизвиква повторно изпълнение на `build()` . Подходящ е само за локално, изолирано в един widget състояние.

5.2 `InheritedWidget`

Вграден механизъм, който позволява данни да се "спускат" надолу по дървото без ръчно подаване през всеки конструктор. Служи като основа, върху която са изградени решения на по-високо ниво като Provider.

5.3 Provider

Пакет, надграждащ `InheritedWidget`, който улеснява инжектирането на зависимости и известяването за промени чрез `ChangeNotifier`:

```
class CartModel extends ChangeNotifier {
  final List<String> _items = [];
  List<String> get items => _items;

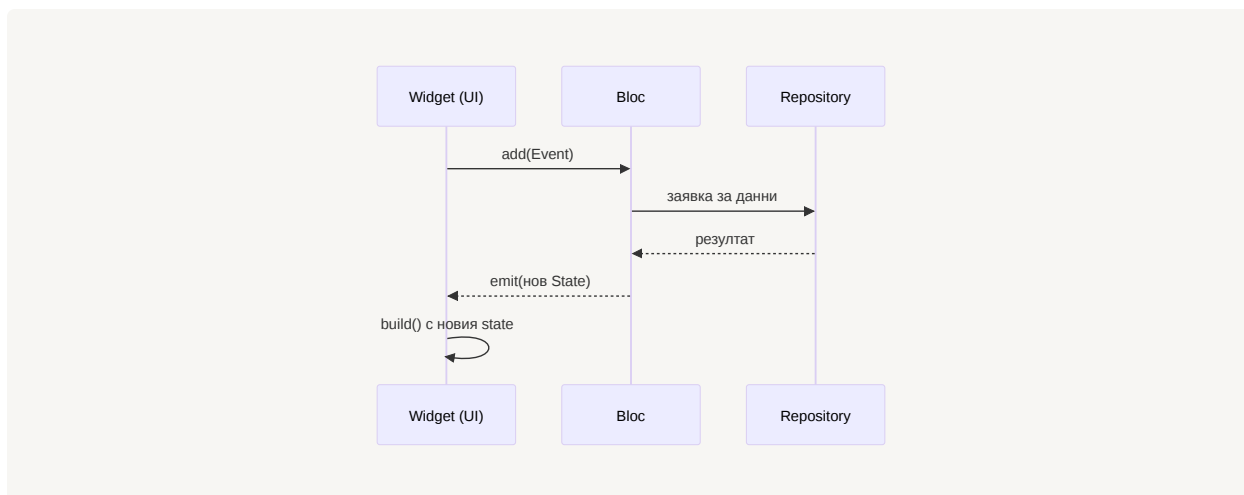
  void add(String item) {
    _items.add(item);
    notifyListeners();
  }
}

// Регистрация:
ChangeNotifierProvider(create: (_) => CartModel(), child: con
st MyApp());

// Употреба в дъщерен widget:
final cart = context.watch<CartModel>();
```

5.4 BLoC (Business Logic Component)

Подход, базиран на потоци (Streams), който строго разделя UI слоя от бизнес логиката: UI изпраща събития (Events), BLoC ги обработва и изпраща нови състояния (States).



5.5 Сравнение на подходите

Подход	Сложност	Подходящ за
setState	ниска	локално, просто състояние в 1 widget
InheritedWidget	средна	вградени решения, основа за библиотеки
Provider	средна	средни по мащаб приложения
Riverpod	средна/висока	compile-safe, тестваеми приложения
BLoC	висока	големи екипи, стриктна архитектура

6. Разработка на мултиплатформени мобилни приложения

6.1 Компиляция и рендиране

За разлика от други cross-platform подходи, Flutter не превежда кода към native UI компоненти на платформата. Вместо това рисува всеки пиксел сам чрез собствен рендиращ енджин - **Skia**, а в по-новите версии - **Impeller**. Това гарантира визуална консистентност между Android и iOS.

За release build Dart кодът се компилира AOT директно в нативен ARM/x64 машинен код - за разлика от режима на разработка, където се използва JIT за бързи hot reload цикли.

6.2 Platform Channels

Когато е нужен достъп до нативни API, недостъпни през Flutter (напр. специфичен сензор), се използва `MethodChannel` за комуникация между Dart и native (Kotlin/Swift) код:

```
static const platform = MethodChannel('com.example.app/batter
y');

Future<int> getBatteryLevel() async {
  final int level = await platform.invokeMethod('getBatteryLe
vel');
  return level;
}
```

6.3 Пакети и pub.dev

Екосистемата pub.dev предоставя хиляди готови пакети (HTTP клиенти, бази данни, state management библиотеки), декларирани в `pubspec.yaml`, което ускорява разработката без нужда от писане на код от нулата.

6.4 Тестване

Flutter поддържа три нива на тестове: **unit tests** (изолирана логика), **widgets tests** (поведение на отделен UI компонент) и **integration tests** (цялостен поток на приложението на реално или симулирано устройство).

6.5 Публикуване

Финалните артефакти се генерират с `flutter build apk` / `appbundle` за Google Play и `flutter build ipa` за App Store, като всяка платформа изисква собствена конфигурация на подписване (signing) и метаданни.